

Lab9 — Battleship Web App

Complete Code Explanation for Oral Exams & Presentations

Table of Contents

1. What is a Servlet? What is JSP?
2. HTTP Basics — Requests, Responses, Headers
3. Project Structure Overview
4. schema.sql — The Database
5. web.xml — Deployment Descriptor
6. DB.java — Database Connection Pool
7. AuthFilter.java — Authentication Gate
8. User.java — Account Logic & Password Hashing
9. Game.java — Game State Machine
10. Ship.java — Data Transfer Object
11. LoginServlet.java
12. RegisterServlet.java
13. LogoutServlet.java
14. JoinServlet.java
15. PlaceServlet.java
16. BombServlet.java
17. StateServlet.java
18. LeaveServlet.java
19. JSP Pages — Frontend
20. The Full Game Flow End-to-End
21. Common Exam Questions & Answers

1. What is a Servlet? What is JSP?

A **Servlet** is a Java class that runs inside a web server (called a *Servlet container*, in our case **Apache Tomcat**). It receives HTTP requests and sends back HTTP responses. Think of it as a function that gets called every time someone visits a URL.

A **JSP (JavaServer Page)** is an HTML file with Java code embedded inside special tags like `<% ... %>`. Tomcat compiles it into a Servlet automatically. JSPs are used for the *view* (what the user sees); Servlets handle the *logic*

(what happens when a form is submitted).

Servlet	JSP
Pure Java class	HTML with embedded Java
Good for processing logic, redirects	Good for rendering HTML pages
Example: LoginServlet.java	Example: login.jsp

The **Servlet lifecycle** is important to know:

1. `init()` — called once when Tomcat loads the servlet. Used for one-time setup (like opening a DB connection).
2. `service()` — called on every request. Internally routes to `doGet()` or `doPost()`.
3. `destroy()` — called once when Tomcat shuts down. Used for cleanup.

2. HTTP Basics — Requests, Responses, Headers

Every time a browser talks to a server it sends an **HTTP request** and receives an **HTTP response**. Both have two parts: *headers* and an optional *body*.

HTTP Methods

Method	Used for	Has body?
GET	Fetching a page or data (read-only)	No
POST	Submitting a form, changing data	Yes

What are Headers?

Headers are key-value pairs sent at the top of every request and response. They tell the browser or server important metadata.

Common request headers the browser sends:

- `Content-Type: application/x-www-form-urlencoded` — tells the server the body is a URL-encoded form (`name=value&name2=value2`).
- `Content-Type: multipart/form-data` — used when a form has file uploads. The body is split into parts.
- `Cookie: JSESSIONID=abc123` — the browser automatically sends the session cookie back to the server.

Common response headers our servlets set:

- `Content-Type: application/json` — tells the browser "the body is JSON, not HTML". Set in `BombServlet` and `StateServlet`.
- `Cache-Control: no-store` — tells the browser never to cache this response. Set in `StateServlet` so polling always gets fresh data.
- `Location: /lab9/lobby.jsp` — set automatically by `sendRedirect()`. Tells the browser to go to a different URL (HTTP 302).

Exam tip: When a Servlet calls `res.setContentType("application/json")`, it is setting the `Content-Type` response header. This is how the browser knows to parse the response as JSON and not try to render it as HTML.

HTTP Status Codes

Code	Meaning	Where used
200 OK	Everything fine	Default when a servlet writes a response

302 Found	Redirect to another URL	<code>sendRedirect()</code>
500 Internal Server Error	Server crashed	SQL exceptions in BombServlet

3. Project Structure Overview

```
lab9/
├── WEB-INF/
│   ├── web.xml          ← Server configuration (filters, session timeout, DB params)
│   ├── lib/
│   │   └── mysql-connector-j-9.7.0.jar ← JDBC driver for MySQL
│   └── classes/         ← Compiled .class files (output of compile.bat)
├── src/lab9/
│   ├── db/DB.java       ← Opens and holds the MySQL connection
│   ├── filters/AuthFilter.java ← Runs on EVERY request; blocks unauthenticated users
│   ├── model/
│   │   ├── Game.java    ← All game logic: join, place ships, bomb, leave
│   │   ├── User.java     ← User accounts, password hashing
│   │   └── Ship.java     ← Simple data object: x, y, sunk
│   └── servlets/
│       ├── LoginServlet.java
│       ├── RegisterServlet.java
│       ├── LogoutServlet.java
│       ├── JoinServlet.java
│       ├── PlaceServlet.java
│       ├── BombServlet.java
│       ├── StateServlet.java
│       └── LeaveServlet.java
├── login.jsp
├── register.jsp
├── lobby.jsp
├── place.jsp
├── game.jsp
├── style.css
├── schema.sql
└── compile.bat
```

The folder `WEB-INF/` is special — Tomcat will **never serve its contents directly** to a browser. This protects your class files, configuration, and libraries from being downloaded by users.

4. schema.sql — The Database

schema.sql

```
CREATE DATABASE IF NOT EXISTS lab9 CHARACTER SET utf8mb4;
USE lab9;

DROP TABLE IF EXISTS bombs;
DROP TABLE IF EXISTS ships;
DROP TABLE IF EXISTS games;
DROP TABLE IF EXISTS users;
```

We drop tables in reverse order of dependency (bombs first, then ships, then games, then users) because of **foreign key constraints**. A foreign key means "this column must reference a valid row in another table." If we tried to drop `users` first, MySQL would refuse because `games` still references it.

```
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  password_hash VARCHAR(128) NOT NULL
);
```

- `AUTO_INCREMENT` — MySQL automatically assigns the next integer ID (1, 2, 3...).
- `UNIQUE` — no two users can have the same username; MySQL enforces this at the database level.
- `password_hash` — we never store the real password, only its SHA-256 hash.

```
CREATE TABLE games (
  id INT AUTO_INCREMENT PRIMARY KEY,
  player1_id INT,
  player2_id INT,
  current_turn INT,
  status VARCHAR(20) NOT NULL, -- WAITING / PLACING / PLAYING / FINISHED
  winner_id INT,
  FOREIGN KEY (player1_id) REFERENCES users(id),
  FOREIGN KEY (player2_id) REFERENCES users(id)
);
```

The `status` column drives the whole game state machine. `current_turn` holds the ID of whichever player should move next (NULL when the game is not in progress).

```
CREATE TABLE ships (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    game_id INT NOT NULL,  
    player_id INT NOT NULL,  
    x INT NOT NULL,  
    y INT NOT NULL,  
    sunk BOOLEAN NOT NULL DEFAULT FALSE,  
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,  
    FOREIGN KEY (player_id) REFERENCES users(id)  
);
```

ON DELETE CASCADE means: if a game row is deleted, MySQL automatically deletes all its ships too. Same for bombs.

```
CREATE TABLE bombs (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    game_id INT NOT NULL,  
    player_id INT NOT NULL,  
    x INT NOT NULL,  
    y INT NOT NULL,  
    hit BOOLEAN NOT NULL,  
    UNIQUE KEY uniq_shot (game_id, player_id, x, y),  
    FOREIGN KEY (game_id) REFERENCES games(id) ON DELETE CASCADE,  
    FOREIGN KEY (player_id) REFERENCES users(id)  
);
```

The **UNIQUE KEY uniq_shot** on (game_id, player_id, x, y) prevents a player from bombing the exact same cell twice — enforced at the database level, not just in Java code.

5. web.xml — Deployment Descriptor

`web.xml` is the configuration file for the entire web application. Tomcat reads it on startup.

WEB-INF/web.xml

```
<session-config>
  <session-timeout>30</session-timeout>
</session-config>
```

Sessions automatically expire after 30 minutes of inactivity. After this, the user is considered logged out.

```
<context-param>
  <param-name>jdbcUrl</param-name>
  <param-value>jdbc:mysql://localhost:3306/lab9?useSSL=false&serverTimezone=UTC</param-
value>
</context-param>
<context-param>
  <param-name>jdbcUser</param-name>
  <param-value>root</param-value>
</context-param>
<context-param>
  <param-name>jdbcPassword</param-name>
  <param-value></param-value>
</context-param>
```

Context parameters are global settings accessible to any servlet or filter. The `DB.java` class reads these three values to build the MySQL connection string. Keeping them in `web.xml` (instead of hardcoding in Java) makes it easy to change the DB without recompiling.

```
<filter>
  <filter-name>auth</filter-name>
  <filter-class>lab9.filters.AuthFilter</filter-class>
</filter>
<filter-mapping>
  <filter-name>auth</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

This registers `AuthFilter` and tells Tomcat to run it for every request (`/*`). A filter is like a middleware — it intercepts requests before they reach the servlet and can block, modify, or pass them through.

6. DB.java — Database Connection

src/lab9/db/DB.java

```
package lab9.db;

import javax.servlet.ServletContext;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DB {
    private static Connection connection;

    public static void init(ServletContext ctx) throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        String url = ctx.getInitParameter("jdbcUrl");
        String user = ctx.getInitParameter("jdbcUser");
        String pass = ctx.getInitParameter("jdbcPassword");
        connection = DriverManager.getConnection(url, user, pass);
    }

    public static Connection get() throws SQLException {
        return connection;
    }
}
```

Line by line:

- `private static Connection connection` — a single, shared MySQL connection for the whole application. This is the **Singleton pattern**: only one instance ever exists.
- `Class.forName("com.mysql.cj.jdbc.Driver")` — loads the MySQL JDBC driver class from the JAR file in `WEB-INF/lib/`. Without this, Java wouldn't know how to talk to MySQL.
- `ctx.getInitParameter("jdbcUrl")` — reads the connection string we defined in `web.xml`. `ctx` is the `ServletContext`, which holds app-wide settings.
- `DriverManager.getConnection(url, user, pass)` — opens the actual TCP connection to MySQL and authenticates. This is the expensive call — we do it once in `init()` and reuse the connection forever.
- `public static Connection get()` — every servlet calls `DB.get()` to get the connection. Because it's `static`, no object needs to be instantiated.

Note: `init()` is called from `AuthFilter.init()` which runs when Tomcat starts. That is why the filter exists — besides blocking unauthenticated users, it initialises the database on startup.

7. AuthFilter.java — Authentication Gate

A **Filter** implements `javax.servlet.Filter` and has three methods: `init()`, `doFilter()`, and `destroy()`. The important one is `doFilter()` — called on every matching request.

src/lab9/filters/AuthFilter.java

```
@WebFilter("/*")
public class AuthFilter implements Filter {

    @Override
    public void init(FilterConfig cfg) throws ServletException {
        try {
            DB.init(cfg.getServletContext());
        } catch (Exception e) {
            throw new ServletException("DB init failed", e);
        }
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                        FilterChain chain) throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse resp = (HttpServletResponse) response;

        String path = req.getServletPath();

        // Allow these paths without a session
        if (path.startsWith("/login") ||
            path.startsWith("/register") ||
            path.endsWith(".css")) {
            chain.doFilter(request, response);
            return;
        }

        HttpSession session = req.getSession(false);
        if (session == null || session.getAttribute("userId") == null) {
            resp.sendRedirect(req.getContextPath() + "/login.jsp");
            return;
        }

        chain.doFilter(request, response);
    }
}
```

Line by line:

- `req.getServletPath()` — gets the path part of the URL, e.g. `/login.jsp` or `/bomb`.
- **Public paths** — login, register, and CSS are allowed through without checking the session. Everything else requires a login.

- `req.getSession(false)` — gets the existing session without creating a new one. If no session exists, returns `null`. The `false` argument is important — `getSession(true)` (the default) would create a new empty session, which we don't want here.
- `session.getAttribute("userId")` — we stored the user's ID in the session at login. If it's not there, the user hasn't logged in.
- `resp.sendRedirect(...)` — sends an HTTP 302 response with a `Location` header. The browser follows it automatically to the login page.
- `chain.doFilter(request, response)` — passes the request along to the next filter or, if there are no more filters, to the actual servlet/JSP. **If you forget this line, the request goes nowhere.**

What is a FilterChain? Filters are arranged in a chain. Each filter can decide to pass the request to the next one (`chain.doFilter()`) or stop it (by writing a response or redirecting). In this app there is only one filter.

8. User.java — Accounts & Password Hashing

src/lab9/model/User.java (key methods)

```
public class User {

    public static String hash(String password) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] digest = md.digest(password.getBytes(StandardCharsets.UTF_8));
            StringBuilder sb = new StringBuilder();
            for (byte b : digest) {
                sb.append(String.format("%02x", b));
            }
            return sb.toString();
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException(e);
        }
    }
}
```

How password hashing works:

We **never store the real password**. Instead, we pass it through a one-way mathematical function (SHA-256) that produces a fixed-length string called a *hash*. You cannot reverse a hash to get the original password.

- `MessageDigest.getInstance("SHA-256")` — gets the SHA-256 algorithm from the Java standard library.
- `md.digest(password.getBytes(...))` — runs the hash function on the password bytes. Returns a 32-byte array.
- The `for` loop — converts each byte to a 2-character hex string (e.g. `0xff` → `"ff"`). The final string is 64 characters long.
- When a user logs in, we hash the submitted password and compare to the stored hash. If they match, the password is correct.

```
public static boolean authenticate(String username, String password)
    throws SQLException {
    try (Connection c = DB.get();
        PreparedStatement ps = c.prepareStatement(
            "SELECT id FROM users WHERE username = ? AND password_hash = ?")) {
        ps.setString(1, username);
        ps.setString(2, hash(password));
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next();
        }
    }
}
```

What is a PreparedStatement?

A `PreparedStatement` is a pre-compiled SQL query with `?` placeholders. You then set the values for each

placeholder using `setString()`, `setInt()`, etc.

Why not just write: `"SELECT id FROM users WHERE username = '" + username + "'"`?

Because if `username` is `' OR '1'='1'`, the query becomes `WHERE username = '' OR '1'='1'` which returns all users — this is a **SQL injection attack**. PreparedStatements prevent this because the value is sent separately from the SQL structure; it can never change the query's logic.

try-with-resources (`try (Connection c = ...) { }`) — Java automatically calls `c.close()` when the block exits, even if an exception is thrown. This prevents connection leaks.

9. Game.java — The State Machine

This is the most important class. It contains all game logic and directly maps to the state transitions in the database.

Game States

Status value	What it means	How you enter it
WAITING	1 player joined, waiting for opponent	First player calls join()
PLACING	Both joined; each must place 2 ships	Second player calls join()
PLAYING	Both placed ships; players take turns	Both players call placeShips()
FINISHED	Game over, winner recorded	Last ship sunk, or someone forfeits

src/lab9/model/Game.java — constants

```
public static final int GRID = 5;
public static final int SHIPS_PER_PLAYER = 2;

public static final String WAITING = "WAITING";
public static final String PLACING = "PLACING";
public static final String PLAYING = "PLAYING";
public static final String FINISHED = "FINISHED";
```

Using constants (instead of typing the string "WAITING" everywhere) means if you ever need to rename a state, you change it in one place and the compiler tells you everywhere it's used.

Game.current()

```
public static Game current() throws SQLException {
    try (Connection c = DB.get();
        PreparedStatement ps = c.prepareStatement(
            "SELECT * FROM games WHERE status <> ? ORDER BY id DESC LIMIT 1")) {
        ps.setString(1, FINISHED);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) return from(rs);
        }
    }
    return null;
}
```

Finds the most recent game that isn't finished. `ORDER BY id DESC LIMIT 1` means "sort by newest first and take only the first row." Returns `null` if no active game exists.

Game.join()

```
public static Game join(int userId) throws SQLException, IllegalStateException {
    Game g = current();
    if (g == null) {
        // No game exists → create one
        try (Connection c = DB.get();
            PreparedStatement ps = c.prepareStatement(
                "INSERT INTO games (player1_id, status) VALUES (?, ?)",
                PreparedStatement.RETURN_GENERATED_KEYS)) {
            ps.setInt(1, userId);
            ps.setString(2, WAITING);
            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                rs.next();
                return byId(rs.getInt(1));
            }
        }
    }
    // Already in this game → return it
    if (g.player1Id != null && g.player1Id == userId) return g;
    if (g.player2Id != null && g.player2Id == userId) return g;

    // Second slot free → join
    if (g.player2Id == null) {
        try (Connection c = DB.get();
            PreparedStatement ps = c.prepareStatement(
                "UPDATE games SET player2_id = ?, status = ? WHERE id = ?")) {
            ps.setInt(1, userId);
            ps.setString(2, PLACING);
            ps.setInt(3, g.id);
            ps.executeUpdate();
        }
        return byId(g.id);
    }
    // Both slots taken → reject
    throw new IllegalStateException("Game is full; a third player cannot join.");
}
```

- `RETURN_GENERATED_KEYS` — when you `INSERT` a row with `AUTO_INCREMENT`, MySQL generates the new ID. This flag tells JDBC to return it so we can read it back.
- `ps.getGeneratedKeys()` — retrieves that auto-generated ID as a `ResultSet`.
- The three-way logic: create → join as p1, join as p2, or reject as p3.
- `IllegalStateException` — a standard Java exception meaning "the object is in the wrong state for this call." `JoinServlet` catches it and shows the error in the lobby.

Game.placeShips() — Transactions

```
public void placeShips(int userId, int[][] cells)
    throws SQLException, IllegalArgumentException {

    // Validation
    if (!PLACING.equals(status))
        throw new IllegalArgumentException("Not in placing phase.");
    if (cells.length != SHIPS_PER_PLAYER)
        throw new IllegalArgumentException("Need exactly 2 ships.");
    for (int[] c : cells) {
        if (c[0] < 0 || c[0] >= GRID || c[1] < 0 || c[1] >= GRID)
            throw new IllegalArgumentException("Ship out of bounds.");
    }
    if (cells[0][0] == cells[1][0] && cells[0][1] == cells[1][1])
        throw new IllegalArgumentException("Ships must be on different cells.");

    try (Connection c = DB.get()) {
        c.setAutoCommit(false);    // START TRANSACTION
        try {
            // Insert ships
            try (PreparedStatement ps = c.prepareStatement(
                "INSERT INTO ships (game_id, player_id, x, y) VALUES (?, ?, ?, ?)") {
                for (int[] cell : cells) {
                    ps.setInt(1, id);
                    ps.setInt(2, userId);
                    ps.setInt(3, cell[0]);
                    ps.setInt(4, cell[1]);
                    ps.executeUpdate();
                }
            }
            // Check if BOTH players have placed
            int p1Count = countShipsOf(c, player1Id);
            int p2Count = countShipsOf(c, player2Id);
            if (p1Count == SHIPS_PER_PLAYER && p2Count == SHIPS_PER_PLAYER) {
                // Transition to PLAYING
                try (PreparedStatement ps = c.prepareStatement(
                    "UPDATE games SET status = ?, current_turn = ? WHERE id = ?") {
                    ps.setString(1, PLAYING);
                    ps.setInt(2, player1Id);    // player1 always goes first
                    ps.setInt(3, id);
                    ps.executeUpdate();
                }
            }
            c.commit();    // COMMIT TRANSACTION
        } catch (Exception e) {
            c.rollback();    // ROLLBACK on any error
            throw e;
        } finally {
            c.setAutoCommit(true);
        }
    }
}
```

What is a database transaction?

A transaction groups multiple SQL statements into an atomic unit: either ALL of them succeed, or NONE of them take effect. We use it here because two things must happen together: insert ships AND (if both are ready) update the game status. If the UPDATE fails after the INSERT, the ships would be in the DB but the game would still be in PLACING state forever. The transaction prevents this — `rollback()` undoes everything back to the previous safe state.

By default, JDBC is in *auto-commit* mode (every statement is its own transaction). We turn it off with `setAutoCommit(false)`, then manually call `commit()` or `rollback()`.

Game.bomb() — Taking a Shot

```
public boolean bomb(int userId, int x, int y)
    throws SQLException, IllegalArgumentException {

    if (!PLAYING.equals(status))
        throw new IllegalArgumentException("Game not in progress.");
    if (currentTurn == null || currentTurn.intValue() != userId)
        throw new IllegalArgumentException("Not your turn.");
    if (x < 0 || x >= GRID || y < 0 || y >= GRID)
        throw new IllegalArgumentException("Shot out of bounds.");

    int opp = opponentOf(userId);

    try (Connection c = DB.get()) {
        c.setAutoCommit(false);
        try {
            // Check for duplicate shot
            try (PreparedStatement ps = c.prepareStatement(
                "SELECT 1 FROM bombs WHERE game_id=? AND player_id=? AND x=? AND y=?")) {
                ps.setInt(1,id); ps.setInt(2,userId); ps.setInt(3,x); ps.setInt(4,y);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) throw new IllegalArgumentException("Already bombed that
cell.");
                }
            }

            // Did we hit a ship?
            boolean hit = false;
            try (PreparedStatement ps = c.prepareStatement(
                "SELECT id FROM ships WHERE game_id=? AND player_id=? AND x=? AND y=?"))
            {
                ps.setInt(1,id); ps.setInt(2,opp); ps.setInt(3,x); ps.setInt(4,y);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        hit = true;
                        int shipId = rs.getInt(1);
                        // Mark ship as sunk
                        try (PreparedStatement up = c.prepareStatement(
                            "UPDATE ships SET sunk = TRUE WHERE id = ?")) {
                            up.setInt(1, shipId);
                            up.executeUpdate();
                        }
                    }
                }
            }

            // Record the bomb
            try (PreparedStatement ps = c.prepareStatement(
                "INSERT INTO bombs (game_id, player_id, x, y, hit) VALUES (?, ?, ?, ?, ?)"))
            {
                ps.setInt(1,id); ps.setInt(2,userId); ps.setInt(3,x);
                ps.setInt(4,y); ps.setBoolean(5,hit);
                ps.executeUpdate();
            }
        }
    }
```

```
// Check if opponent has any ships left
int remaining;
try (PreparedStatement ps = c.prepareStatement(
    "SELECT COUNT(*) FROM ships WHERE game_id=? AND player_id=? AND
sunk=FALSE")) {
    ps.setInt(1,id); ps.setInt(2,opp);
    try (ResultSet rs = ps.executeQuery()) {
        rs.next(); remaining = rs.getInt(1);
    }
}

if (remaining == 0) {
    // Game over - current player wins
    try (PreparedStatement ps = c.prepareStatement(
        "UPDATE games SET status=?, winner_id=?, current_turn=NULL WHERE
id=?")) {
        ps.setString(1, FINISHED);
        ps.setInt(2, userId);
        ps.setInt(3, id);
        ps.executeUpdate();
    }
} else {
    // Advance turn to opponent
    try (PreparedStatement ps = c.prepareStatement(
        "UPDATE games SET current_turn=? WHERE id=?")) {
        ps.setInt(1, opp);
        ps.setInt(2, id);
        ps.executeUpdate();
    }
}
c.commit();
return hit;
} catch (Exception e) {
    c.rollback();
    throw e;
} finally {
    c.setAutoCommit(true);
}
}
```

The entire shot is wrapped in one transaction. If any SQL fails in the middle, everything is rolled back and no partial state is saved. The method returns `true` if the shot was a hit, `false` if it was a miss.

10. Ship.java — Data Transfer Object

```
public class Ship {  
    public final int x, y;  
    public final boolean sunk;  
  
    public Ship(int x, int y, boolean sunk) {  
        this.x = x;  
        this.y = y;  
        this.sunk = sunk;  
    }  
}
```

A **DTO (Data Transfer Object)** is a simple class with no logic — just fields. Its only purpose is to carry data between layers (e.g. from the database query in `Game.java` to the JSON serializer in `StateServlet.java`). The `final` keyword means these fields cannot be changed after the object is created.

11. LoginServlet.java

src/lab9/servlets/LoginServlet.java

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException, ServletException {
        req.getRequestDispatcher("/login.jsp").forward(req, resp);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws IOException, ServletException {
        String username = req.getParameter("username");
        String password = req.getParameter("password");
        try {
            if (User.authenticate(username, password)) {
                int userId = User.findByUsername(username);
                HttpSession session = req.getSession(true); // create new session
                session.setAttribute("userId", userId);
                session.setAttribute("username", username);
                resp.sendRedirect(req.getContextPath() + "/lobby.jsp");
            } else {
                req.setAttribute("error", "Invalid username or password.");
                req.getRequestDispatcher("/login.jsp").forward(req, resp);
            }
        } catch (SQLException e) {
            throw new ServletException(e);
        }
    }
}
```

- `@WebServlet("/login")` — this annotation tells Tomcat to map the URL `/login` to this class. No `web.xml` entry needed for the servlet itself (only filters need `web.xml` in this project).
- `doGet()` — when the user navigates to `/login`, we just show the login form by forwarding to `login.jsp`. A *forward* happens server-side — the browser URL doesn't change.
- `doPost()` — handles the form submission. `req.getParameter("username")` reads the value of the form field with `name="username"`.
- `req.getSession(true)` — creates a brand new session. Tomcat generates a unique ID (`JSESSIONID`) and sends it to the browser as a cookie. On every subsequent request, the browser sends this cookie back so Tomcat can find the right session object.
- `session.setAttribute("userId", userId)` — stores the user's ID in the session so other servlets can identify who is making requests.
- `sendRedirect` vs `forward`: `redirect` sends HTTP 302 (browser navigates to new URL), `forward` happens

silently on the server.

12. RegisterServlet.java

```
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws IOException, ServletException {
        String username = req.getParameter("username");
        String password = req.getParameter("password");
        String confirm = req.getParameter("confirm");

        // Client-side validation mirrored server-side
        if (username == null || !username.matches("[A-Za-z0-9_]{3,30}")) {
            req.setAttribute("error", "Username must be 3-30 characters (letters, digits,
underscore).");
            req.getRequestDispatcher("/register.jsp").forward(req, resp);
            return;
        }
        if (password == null || password.length() < 4) {
            req.setAttribute("error", "Password must be at least 4 characters.");
            req.getRequestDispatcher("/register.jsp").forward(req, resp);
            return;
        }
        if (!password.equals(confirm)) {
            req.setAttribute("error", "Passwords do not match.");
            req.getRequestDispatcher("/register.jsp").forward(req, resp);
            return;
        }
        try {
            User.create(username, password);
            int userId = User.findByUsername(username);
            HttpSession session = req.getSession(true);
            session.setAttribute("userId", userId);
            session.setAttribute("username", username);
            resp.sendRedirect(req.getContextPath() + "/lobby.jsp");
        } catch (SQLException e) {
            req.setAttribute("error", "Username already taken.");
            req.getRequestDispatcher("/register.jsp").forward(req, resp);
        }
    }
}
```

- `username.matches("[A-Za-z0-9_]{3,30}")` — a **regular expression**. It means: only letters (A-Z, a-z), digits (0-9), and underscores (`_`), between 3 and 30 characters long. The whole string must match (implicit anchoring with `matches()`).
- After a successful registration, the user is automatically logged in — a good UX practice so they don't have to log in immediately after registering.
- The `catch (SQLException e)` at the bottom catches the MySQL unique constraint violation — if the username is already taken, MySQL throws an exception which we translate into a friendly error message.

13. LogoutServlet.java

```
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) {
            session.invalidate();
        }
        resp.sendRedirect(req.getContextPath() + "/login.jsp");
    }
}
```

`session.invalidate()` destroys the session on the server side. Tomcat also tells the browser to delete the `JSESSIONID` cookie. After this, the user is effectively logged out — any further requests will have no session and `AuthFilter` will redirect them to login.

14. JoinServlet.java

```
@WebServlet("/join")
public class JoinServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        Integer userId = (Integer) req.getSession().getAttribute("userId");
        try {
            Game g = Game.join(userId);
            req.getSession().setAttribute("gameId", g.id);
            res.sendRedirect(req.getContextPath() + "/lobby.jsp");
        } catch (IllegalStateException e) {
            req.setAttribute("error", e.getMessage());
            req.getRequestDispatcher("/lobby.jsp").forward(req, res);
        } catch (SQLException e) {
            throw new ServletException(e);
        }
    }
}
```

Reads `userId` from the session, asks `Game.join()` to add this player to a game, then stores the returned `gameId` in the session. If the game is full, `Game.join()` throws `IllegalStateException` which is caught here and shown as an error in the lobby.

15. PlaceServlet.java

```
@WebServlet("/place")
public class PlaceServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        Integer userId = (Integer) req.getSession().getAttribute("userId");
        Integer gameId = (Integer) req.getSession().getAttribute("gameId");
        try {
            Game g = Game.byId(gameId);
            int x0 = Integer.parseInt(req.getParameter("x0"));
            int y0 = Integer.parseInt(req.getParameter("y0"));
            int x1 = Integer.parseInt(req.getParameter("x1"));
            int y1 = Integer.parseInt(req.getParameter("y1"));
            g.placeShips(userId, new int[][]{{x0, y0}, {x1, y1}});
            resp.sendRedirect(req.getContextPath() + "/place.jsp");
        } catch (IllegalArgumentException e) {
            req.setAttribute("error", e.getMessage());
            req.getRequestDispatcher("/place.jsp").forward(req, resp);
        } catch (SQLException | NumberFormatException e) {
            throw new ServletException(e);
        }
    }
}
```

Reads the four coordinate parameters (`x0`, `y0` for ship 1 and `x1`, `y1` for ship 2) sent from the `place.jsp` form. Passes them to `Game.placeShips()` which does all the validation and DB work. On success, redirects back to `place.jsp` (which will poll `/state` and redirect to `game.jsp` once both players have placed).

16. BombServlet.java

src/lab9/servlets/BombServlet.java

```
@WebServlet("/bomb")
public class BombServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.setContentType("application/json");
        Integer userId = (Integer) req.getSession().getAttribute("userId");
        try (PrintWriter w = res.getWriter()) {
            try {
                Game g = Game.current();
                if (g == null) {
                    json(w, "{\"ok\":false,\"error\":\"No active game\"}");
                    return;
                }
                int x = Integer.parseInt(req.getParameter("x"));
                int y = Integer.parseInt(req.getParameter("y"));
                boolean hit = g.bomb(userId, x, y);
                json(w, "{\"ok\":true,\"hit\":" + hit + "}");
            } catch (NumberFormatException e) {
                json(w, "{\"ok\":false,\"error\":\"Invalid coordinates\"}");
            } catch (IllegalArgumentException e) {
                json(w, "{\"ok\":false,\"error\":\"" + escape(e.getMessage()) + "\"}");
            } catch (SQLException e) {
                res.setStatus(500);
                json(w, "{\"ok\":false,\"error\":\"Server error\"}");
            }
        }
    }

    private void json(PrintWriter w, String s) { w.print(s); }

    private String escape(String s) {
        if (s == null) return "\"\"";
        return "\"" + s.replace("\\", "\\\").replace("\"", "\\\"") + "\"";
    }
}
```

- `res.setContentType("application/json")` — sets the `Content-Type` header. This is the first thing we do so the browser knows to parse our response as JSON.
- The response is always a JSON object with an `ok` boolean. This is a common API pattern — the JavaScript on the client checks `if (!j.ok)` to decide if something went wrong.
- `escape()` — escapes backslashes and double quotes inside the error message before embedding it in JSON. Without this, a message like `You said "hello"` would produce invalid JSON.

Important fix — FormData vs URLSearchParams:

The JavaScript fetch call uses `new URLSearchParams()` to build the request body. This sends the data as `application/x-www-form-urlencoded` (like a regular HTML form), which means `req.getParameter("x")` works correctly in the servlet.

If we had used `new FormData()` instead, it would send `multipart/form-data`. Tomcat only parses multipart bodies if the servlet is annotated with `@MultipartConfig`. Without that annotation, `req.getParameter("x")` would return `null`, causing a `NumberFormatException` — which is exactly the "Invalid coordinates" bug that was fixed.

17. StateServlet.java

This is the most-called servlet — every game page polls it every 2 seconds via AJAX.

src/lab9/servlets/StateServlet.java

```
@WebServlet("/state")
public class StateServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws IOException {
        res.setContentType("application/json");
        res.setHeader("Cache-Control", "no-store");
        Integer userId = (Integer) req.getSession().getAttribute("userId");
        try (PrintWriter w = res.getWriter()) {
            try {
                Game g = Game.current();
                if (g == null) {
                    w.print("{\"status\":\"NONE\"}");
                    return;
                }
                StringBuilder sb = new StringBuilder();
                sb.append("{");
                sb.append("\"gameId\":").append(g.id);
                sb.append(", \"status\":").append(g.status).append("\"");
                sb.append(", \"grid\":").append(Game.GRID);
                sb.append(", \"isPlayer\":").append(g.hasPlayer(userId));
                sb.append(", \"yourTurn\":")
                    .append(g.currentTurn != null
                        && g.currentTurn.intValue() == userId.intValue());
                sb.append(", \"winner\":").append(name(g.winnerId));
                // ... ships and bombs appended if isPlayer
                sb.append("}");
                w.print(sb.toString());
            } catch (SQLException e) {
                res.setStatus(500);
                w.print("{\"error\":\"Server error\"}");
            }
        }
    }
}
```

- `res.setHeader("Cache-Control", "no-store")` — sets an HTTP response header that tells the browser and any proxy caches to never save this response. This is critical for polling: without it, the browser might return a cached copy and the player would never see updates.
- `StringBuilder` — builds the JSON string efficiently by appending parts. More efficient than string concatenation with `+` inside a loop (which creates a new `String` object on every concatenation).
- `g.currentTurn.intValue() == userId.intValue()` — comparing two `Integer` objects with `==` compares their memory addresses, not their values (a Java gotcha). Calling `.intValue()` unboxes them to `int`

primitives so value comparison works correctly.

- The JSON returned includes: status, grid size, whether the current user is a player, whose turn it is, ship positions, and bomb history.

18. LeaveServlet.java

```
@WebServlet("/leave")
public class LeaveServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
        HttpSession session = req.getSession(false);
        if (session == null) {
            resp.sendRedirect(req.getContextPath() + "/login.jsp");
            return;
        }
        Integer userId = (Integer) session.getAttribute("userId");
        Integer gameId = (Integer) session.getAttribute("gameId");
        if (userId != null && gameId != null) {
            try {
                Game g = Game.byId(gameId);
                if (g != null && g.hasPlayer(userId)) g.leave(userId);
            } catch (Exception ignored) {}
            session.removeAttribute("gameId");
        }
        resp.sendRedirect(req.getContextPath() + "/lobby.jsp");
    }
}
```

When a player forfeits or cancels placement:

1. If the game was WAITING (only one player), the game row is deleted from the database.
2. If the game was PLACING or PLAYING, it is marked FINISHED and the other player wins.
3. `session.removeAttribute("gameId")` — clears the game reference from the session so the lobby starts fresh.
4. Exceptions are swallowed (`catch (Exception ignored)`) because even if the DB call fails, we still want to redirect the user to the lobby.

19. JSP Pages — Frontend

login.jsp

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%
    String error = (String) request.getAttribute("error");
%>
<!DOCTYPE html>
...
<% if (error != null) { %>
    <div class="error"><%= error %></div>
<% } %>
<form method="post" action="${pageContext.request.contextPath}/login">
    <input type="text" name="username" required minlength="3" maxlength="30">
    <input type="password" name="password" required>
    <button type="submit">Log in</button>
</form>
```

- `<%@ page ... %>` — a *JSP page directive*. Sets the content type and encoding for the whole page.
- `<% ... %>` — a *JSP scriptlet*. Regular Java code executed on the server before the page is sent to the browser.
- `<%= error %>` — a *JSP expression*. Outputs the value of `error` into the HTML. Equivalent to `out.print(error)`.
- `${pageContext.request.contextPath}` — *Expression Language (EL)*. Returns the app's context path (e.g. `/lab9`). Makes links work correctly regardless of where the app is deployed.
- `required minlength="3" maxlength="30"` — *HTML5 client-side validation*. The browser checks these before submitting the form. The server also validates them independently (defence in depth).

lobby.jsp — Polling with AJAX

```

<script>
  var ctx = '${pageContext.request.contextPath}';

  function render(s) {
    var status = document.getElementById('status');
    var actions = document.getElementById('actions');

    if (!s.status || s.status === 'NONE') {
      status.textContent = 'No active game. Start one!';
      actions.innerHTML = joinForm('Start a new game');
      return;
    }
    if (s.status === 'FINISHED') {
      status.innerHTML = 'Last game finished. Winner: <strong>' +
        (s.winner || '-') + '</strong>';
      actions.innerHTML = joinForm('Start a new game');
      return;
    }
    if (s.isPlayer && s.status === 'PLACING') {
      window.location.href = ctx + '/place.jsp';
      return;
    }
    if (s.isPlayer && s.status === 'PLAYING') {
      window.location.href = ctx + '/game.jsp';
      return;
    }
    ...
  }

  function poll() {
    fetch(ctx + '/state', { credentials: 'same-origin' })
      .then(function(r) { return r.json(); })
      .then(render)
      .catch(function() { document.getElementById('status').textContent = 'Connection
error.'; });
  }

  poll();
  setInterval(poll, 2000);
</script>

```

- `fetch()` — a modern browser API for making HTTP requests from JavaScript without reloading the page. This is **AJAX** (Asynchronous JavaScript And XML — though we use JSON, not XML).
- `credentials: 'same-origin'` — tells `fetch` to include the session cookie (`JSESSIONID`) in the request. Without this, the server wouldn't know who is asking and would treat it as unauthenticated.
- `.then(function(r) { return r.json(); })` — `fetch` is *asynchronous* (it doesn't block the page). `.then()` registers a callback that runs when the server responds. `r.json()` parses the response body as JSON and returns another Promise.
- `.then(render)` — once the JSON is parsed, calls `render(s)` with the parsed object.
- `setInterval(poll, 2000)` — calls `poll()` every 2000 milliseconds (2 seconds). This is how the page stays

up-to-date with the game state without the user pressing refresh.

- The `render()` function reads the `status` field and decides what to show. If the game advances to PLACING or PLAYING, `window.location.href` automatically navigates the user to the right page.

place.jsp — Ship Placement Grid

```
<script>
var ctx  = '${pageContext.request.contextPath}';
var GRID = 5;
var selected = [];

// Build the grid dynamically
var board = document.getElementById('board');
board.style.gridTemplateColumns = 'repeat(' + GRID + ', 44px)';
for (var y = 0; y < GRID; y++) {
    for (var x = 0; x < GRID; x++) {
        var cell = document.createElement('button');
        cell.type = 'button';
        cell.dataset.x = x;
        cell.dataset.y = y;
        cell.addEventListener('click', onSelect);
        board.appendChild(cell);
    }
}

function onSelect() {
    var x = parseInt(this.dataset.x);
    var y = parseInt(this.dataset.y);
    // Toggle selection
    var idx = selected.findIndex(function(s) { return s.x===x && s.y===y; });
    if (idx >= 0) {
        selected.splice(idx, 1);
        this.classList.remove('selected');
    } else if (selected.length < 2) {
        selected.push({x:x, y:y});
        this.classList.add('selected');
    }
    // Enable confirm only when 2 cells selected
    document.getElementById('confirmBtn').disabled = selected.length !== 2;
}
</script>
```

- `cell.dataset.x = x` — stores the x coordinate in the button's `data-x` HTML attribute. When the button is clicked, we can read `this.dataset.x` to know which cell was clicked.
- `selected.findIndex()` — searches the array for a cell with the same coordinates. If found, clicking again deselects it (toggling behavior).
- `selected.splice(idx, 1)` — removes 1 element at position `idx` from the array.
- The confirm button stays disabled until exactly 2 cells are selected — enforced in the browser by the `disabled` property.

game.jsp — The Bomb Board

```
function onBomb() {
  if (!canBomb || gameOver) return;
  if (this.classList.contains('hit') ||
      this.classList.contains('miss') ||
      this.classList.contains('sunk')) return;

  var x = this.dataset.x;
  var y = this.dataset.y;
  var params = new URLSearchParams();
  params.append('x', x);
  params.append('y', y);
  fetch(ctx + '/bomb', { method: 'POST', body: params, credentials: 'same-origin' })
    .then(function(r) { return r.json(); })
    .then(function(j) {
      if (!j.ok) { alert(j.error || 'Move rejected'); }
      poll();
    });
}
```

- `canBomb` — a flag set by `render()` based on `s.yourTurn`. If it's not your turn, clicking does nothing.
- The `classList.contains()` checks prevent clicking on a cell you've already bombed.
- `new URLSearchParams()` — builds a URL-encoded form body (`x=2&y=3`). This is important: it sends as `application/x-www-form-urlencoded` so the servlet can read the parameters with `req.getParameter()`.
- After the shot, `poll()` is called immediately to refresh the board — no need to wait for the next 2-second interval.

20. The Full Game Flow End-to-End

Step	Who does it	What happens
1. Register / Login	Player 1 & 2	Session created with userId + username
2. Click "Start a new game"	Player 1	POST /join → Game created (WAITING) → gameId in session
3. Lobby polls /state	Both	Player 1 sees "waiting"; Player 2 sees "join" button
4. Click "Join as player 2"	Player 2	POST /join → game updated (PLACING) → both redirected to place.jsp
5. Select 2 cells, click Confirm	Both	POST /place → ships inserted in DB; when both done → game → PLAYING
6. game.jsp loads	Both	Boards built from /state; Player 1 has canBomb=true
7. Click opponent cell	Player 1	POST /bomb → hit/miss recorded → turn switches to Player 2
8. Repeat turns	Both	Each poll refreshes both boards; sunk ships shown in dark red
9. Last ship sunk	Winner	Game → FINISHED, winner_id set; "Game over" shown, forfeit button hidden
10. Forfeit mid-game	Either	POST /leave → game → FINISHED, opponent wins → redirect lobby

21. Common Exam Questions & Answers

Q: What is a session and how does it work in Java Servlets?

A session is a server-side object (`HttpSession`) that stores data between multiple requests from the same browser. When a session is created, Tomcat generates a unique random ID (`JSESSIONID`) and sends it to the browser as a cookie. On every subsequent request, the browser sends this cookie back, allowing Tomcat to find the correct session object and retrieve stored attributes like `userId`.

Q: What is the difference between `sendRedirect` and `forward`?

- `sendRedirect(url)` — sends an HTTP 302 response to the browser, which then makes a new GET request to the given URL. The browser's address bar changes. Used in `LoginServlet` after successful login.
- `forward(req, resp)` — the server internally passes the request to another resource (JSP/Servlet) without the browser knowing. The URL stays the same. Used when showing an error on the same form (the request attributes like "error" are preserved).

Q: Why do we use `PreparedStatement` instead of string concatenation for SQL?

To prevent SQL injection. If we concatenated user input directly into SQL strings, a malicious user could input something like `' OR '1'='1` to bypass authentication or `'; DROP TABLE users; --` to delete data.

`PreparedStatement` separate the SQL structure from the data values — the database never interprets user input as SQL code.

Q: What does `Content-Type: application/json` do?

It is an HTTP response header that tells the browser what format the response body is in. When `BombServlet` sets `res.setContentType("application/json")`, it tells the browser (and the JavaScript `fetch` call) that the response is JSON data, not an HTML page. Without this header, the browser might try to display it as a webpage.

Q: What is `Cache-Control: no-store` and why is it set in `StateServlet`?

It is an HTTP response header that tells the browser (and any intermediate proxy servers) never to store a cached copy of this response. `StateServlet` is polled every 2 seconds to get the current game state — if the browser cached the response, every poll would return the same stale data and the game would appear frozen. `no-store` forces the browser to always ask the server for a fresh copy.

Q: What is a Filter and how is it different from a Servlet?

A Servlet handles a specific URL. A Filter intercepts requests to multiple URLs (in our case all URLs via `/*`) and can pre-process or block them before they reach the servlet. `AuthFilter` checks for a valid session before any request reaches any servlet. If no session exists, it redirects to login — the actual servlet never runs.

Q: What is a database transaction and why is it used in `placeShips()` and `bomb()`?

A transaction groups multiple SQL statements so that either all succeed or none of them take effect (atomicity). In `placeShips()`, we insert ships AND update the game status in the same transaction. If the status update failed, the ships would be in the database but the game would be stuck in PLACING forever. The transaction's rollback prevents this partial state. The same applies to `bomb()` where we record the shot, mark ships sunk, and advance the turn — all of which must happen together.

Q: What is AJAX and how is it used here?

AJAX (Asynchronous JavaScript And XML) means making HTTP requests from JavaScript without reloading the whole page. In this app, the JavaScript `fetch()` API calls `/state` every 2 seconds. The server returns JSON describing the current game state, and JavaScript updates only the parts of the page that changed (the board, status message) — without a full page reload. This gives the appearance of real-time multiplayer updates.

Q: Why does the app use URLSearchParams instead of FormData in the bomb fetch call?

`FormData` sends data as `multipart/form-data`, which Tomcat only parses if the servlet has the `@MultipartConfig` annotation. Without it, `req.getParameter("x")` returns `null`, causing a `NumberFormatException`. `URLSearchParams` sends as `application/x-www-form-urlencoded` (the same as a regular HTML form), which Tomcat always parses. So `req.getParameter("x")` correctly returns the coordinate value.

Q: How does password hashing work and why don't we store plain passwords?

SHA-256 is a one-way mathematical function — given any input, it produces a fixed 64-character output (hash). It is impossible to reverse the hash back to the original password. We store only the hash. When a user logs in, we hash their submitted password and compare it to the stored hash. If they match, the password is correct. This way, even if the database is stolen, attackers cannot recover user passwords.

Q: What happens when a third player tries to join?

The lobby polls `/state` every 2 seconds. If a game is in PLAYING or PLACING state and the current user is not one of the two players (`isPlayer: false`), the lobby shows "A game is already in progress... You cannot join until it ends." with no join button. If they somehow POST directly to `/join`, `Game.join()` reaches the line `throw new IllegalStateException("Game is full; a third player cannot join.")`, which `JoinServlet` catches and displays as an error in the lobby.